

Temă: Document Analyst cu RAG

Construiește pipeline-ul de extracție + storage cu pgvector și conectează-l la agentul din L1-L2

Flow așteptat:



```
1 # pipeline.py - load -> chunk -> extract -> store
2 def process(file: Path) -> Document:
3     docs = load(file)    # registry pe extensie
4     chunks = split(docs, 800, 100)
5     data = llm.with_structured_output(Invoice)
6     emb = embed(chunks)  # sentence-transformers
7     return repo.save(data, chunks, emb)
8
9 # agent.py - tool RAG conectat la LLM-ul din L1-L2
10 @tool
11 def search_documents(query: str) -> str:
12     return rag.search(query, top_k=3)
13
14 agent.invoke("Ce clauze de reziliere avem?")
```

Ce trebuie făcut:

1

Extraction Pipeline (L3)

- Loader cu registry pentru PDF / DOCX / TXT
- Chunking + schemă Pydantic (factură sau contract)
- Pipeline: load -> chunk -> extract + salvare JSON

2

PostgreSQL + Repository Pattern (L4)

- Postgres cu pgvector via Docker + Alembic migrations
- Modele Document + DocumentChunk (one-to-many)
- Repository pentru CRUD și transaction management

3

RAG cu Embeddings (L4)

- Embeddings cu sentence-transformers per chunk
- Similarity search cu cosine distance + HNSW index
- RAGService.search(query, top_k) funcțional

4

Conectare la Agentul din L1-L2

- Wrap RAG ca LangChain tool: search_documents()
- Adaugă tool-ul la agentul existent
- Demo: agent răspunde pe baza documentelor încărcate